

Itch: A package manager for Linux From Scratch

Shayan Nezami

September 10, 2025

Contents

1	Introduction	2
1.1	Background	2
1.1.1	Linux	2
1.1.2	Package managers	2
1.1.3	Linux From Scratch	3
1.2	Motivation	3
2	Design	4
2.1	Non-technical aims	4
2.2	Package maintenance	5
2.2.1	Modifying destdir	5
2.2.2	Using unionfs	5
2.2.3	Functional package managers	5
2.2.4	Conclusion	5
2.3	Dependency resolution	5
2.3.1	Maintaining a curated repository	5
2.3.2	Holding a package database	6
3	Programming the package manager	6

1 Introduction

1.1 Background

In this section, I will set out some key terms and definitions, and explain some crucial concepts, knowledge of which form prerequisites to the rest of the paper. This is intended mostly towards readers with little knowledge about modern Linux systems.

1.1.1 Linux

Linux, while sometimes referred to as an operating system, like macOS and Windows, is in fact just a kernel. A **kernel** is, in simple terms, the core of an operating system, dealing with the CPU, and allocating memory to processes, among other essential operations. However, on its own, it does not provide a functional system, and is certainly not an operating system itself [1].

An operating system is a collection of software that provides a usable system to an end user. Therefore, the Linux kernel is often distributed along with large amounts of other software, in the form of a **Linux distribution**. There are hundreds, if not thousands, of such distributions, examples including Ubuntu, Debian, and Arch Linux. When I refer to a **Linux system**, this is not in reference to the Linux kernel, but rather a full operating system built on top of the kernel.

1.1.2 Package managers

All computer systems need software to function, so there must be a mechanism for users to install software. At its core, **software** is simply code, that is **compiled** ("translated") to a binary format that can be executed by the processor. Installing software, in its most basic form, just consists of copying those binary files to a location where the system recognises them as executable programs.

For Windows systems, most software comes with a dedicated installer (and uninstaller), which carries out this process for the application. However, this is very rarely the case on Linux systems. Software is often released as just source code, and the user is responsible for the installation.

Manual installation is not particularly difficult (the code simply needs to be downloaded, compiled, tested, then copied into place), however this can get tedious when repeated for hundreds of programs. Furthermore, this makes uninstallation very difficult, as there is no way to know which files were installed by which program, and dependency management becomes near impossible.

Dependencies are software or libraries that are required for a program to be installed. There are also **conflicts** between software, where two programs cannot be simultaneously installed on a system. Often, these occur between versions of packages.

For example, where p denotes a package, p_a may require p_c with version ≥ 3.0 , and p_b may conflict with p_c versions ≥ 3.5 . This means that for p_a, p_b, p_c to be simultaneously installed, p_c must have version v , where $3.0 \leq v < 3.5$.

Therefore, one of the defining features of a Linux distribution is the package manager they include. **Package managers** can perform all the tasks above, and more. Very often they have access to a repository of trusted and popular software, so the user can install, uninstall, and update software with just one command. For example, the following command in Arch Linux installs the web browser, Firefox:

```
# pacman -S firefox
```

Here, pacman - the package manager for Arch Linux [2] - gets the *package* for Firefox from its public repository, and then copies the required files into place, keeping records of what files it installed and where. This incredible ease of use is a massive draw towards Linux, and specifically existing Linux distributions.

1.1.3 Linux From Scratch

Linux From Scratch (LFS) is a project that provides an online, continuously updated, book with instructions on how to build a Linux system yourself, compiling and installing all software manually from source code [3], and not using any of the provided distributions - the vast majority of which come with some form of installer.

1.2 Motivation

Having recently installed a Linux From Scratch system, there is a clear gap for a package manager, which is not provided in the book. This is intentional, to bring the user's attention to compiling and installing the base software from scratch [4]. However, this is not sustainable if the system is to become usable long-term.

An advanced, practically usable system will at some point need complex software packages, including but certainly not limited to graphical toolchains, programming frameworks, and desktop environments. These come with many dependencies, shared libraries, and version conflicts.

Even without such large packages, as a system grows, so will the number of packages (even smaller ones) it has, which feed into the same issues. There are also problems associated with upgrading software, where all files relating to the old version need to be cleanly uninstalled, and replaced with the new version, without breaking the dependencies of any other currently installed software [4]. It is not sustainable to continue without a well-defined package manager at this point.

This drives users into three main groups:

1. They create a basic, personal package manager to suit their needs.
2. They install another distribution's package manager, such as Gentoo's Portage, or Arch's pacman [5].
3. They leave their LFS system as-is after installation, never using it for any practical purpose, rather leaving it as a completed educational experience.

All three of these come with significant, non-trivial downsides.

In the first case, these package managers tend to be simple and limited in scope, and while their creation does provide a good educational experience, they are often not capable of fully supporting a growing system.

Plus, programming ability is not a prerequisite to building an LFS system, so, while a minority, there are users who are capable of installing such a system, but not creating a package manager for it. While there are no formal studies on the proportion of Linux users who are not programmers, online discussions seem to suggest the existence of a notable minority [6].

The second case has the benefit of providing a highly reliable, popular, and documented package manager, often with all the user's needed features and more. However, there are two key downsides here.

Firstly, this in effect turns the user's LFS system into a version of the distribution providing the package manager, eliminates many of the purposes of LFS, which is about creating and maintaining your own

system, with full control over all software installed. The sheer complexity of these package managers removes such control from the users, and while it does often produce a working system, the user has in effect just followed a very long path to installing that distribution.

Secondly, and perhaps most crucially, these package managers have been specifically designed for their own distribution, and an LFS system is by definition different. These systems may handle certain cases differently, or have different directory structures, among other subtle differences. Since the package manager has not been made for, and is not commonly used with, LFS systems, there tends to be a lack of support, with a very small user base that can help with any issues that will eventually arise.

For these reasons, I came to the decision to write a package manager, dedicated to Linux From Scratch systems, addressing all the shortfalls identified.

2 Design

2.1 Non-technical aims

I will start by defining some fundamental aims of the package manager that will be produced. Many will seem trivial, but it is important to set these out to guide the technical design decisions.

1. It must be targetted towards LFS systems. While it should ideally be usable on most Linux systems, decisions should be guided by what is needed on an LFS system, and what the LFS user base would prefer.
2. The compilation ("building") of software should be left to the user, meaning precompiled binaries should never be distributed. Linux From Scratch is based on the principles of compiling ones own software, and this is a key requirement if a package manager is to be used by LFS users.
3. Users should be able to patch and modify source code as they desire prior to installation, with the package manager placing no restriction on this.
4. The compiled software should be stored as *packages* of some form for future use, so that the user does not have to rebuild software to reinstall it. Note that this does not defy Aim 2, since these packages will have been compiled by the user themselves, configured as they desire, rather than externally compiled, then distributed.
5. There should be no central server the package manager needs to interact with to function. It should be able to run wholly locally, with no limitations.
6. It must provide a method for full and clean uninstallation, removing all traces of the software, while not affecting any other programs.
7. Dependencies and conflicts must be tracked and managed appropriately, as a key purpose of this project.
8. It must not simply be a derivative of an existing package manager - there must be something unique about it, or a non-trivial reason to use it instead of another existing package manager. This has been partially addressed in Section 1.2, but it should be continuously thought about.

I chose the name *Itch* for this package manager to reflect the importance of Aim 1, since it is being built for Linux From *Scratch*.

2.2 Package maintenance

Through my research of existing package managers, I have identified three promising methods employed by package managers to install and subsequently uninstall packages. These generally all provide some method to track what files are installed, and where, so that they can be removed on uninstallation.

These are generally independent of the method for dependency resolution, which will be discussed in Section 2.3.

2.2.1 Modifying `destdir`

2.2.2 Using `unionfs`

2.2.3 Functional package managers

2.2.4 Conclusion

... so I have chosen to use the `destdir` method, where ...

2.3 Dependency resolution

Now, I must consider the problem of dependency resolution. Each package can depend on, or conflict with, certain other packages, meaning certain packages must be installed with each other, while other packages cannot coexist on a system. To make this more complex, these are most often bounded by versions. See Section 1.1.2 for a deeper explanation.

This means that package managers must come with a method to *resolve* dependencies, meaning to find what packages - and in what versions - need to be installed on the system for a new package to be added.

Dependency resolution is an NP-complete problem [7], meaning in practical terms that it cannot reliably be solved "quickly" (there is no known polynomial time solution). Therefore, shortcuts and optimisations are often taken to make it tractable.

2.3.1 Maintaining a curated repository

The vast majority of popular Linux distributions employ this method, where the maintainers of the distribution maintain an online repository, containing one version of each package available, ensuring that those versions all work together. This greatly simplifies the problem of dependency resolution, since package versions are no longer a concern.

Another reason this method is very commonly used in more mainstream distributions is that the package set becomes universal among systems of the same version, making problems easier to diagnose and debug.

Moreover, this fits very well with the philosophies of some distributions. For example, Arch Linux operates on a rolling-release model, always using the very newest software, so this curated repository only contains the newest usable builds [8]. On the opposite end of the spectrum, Debian focuses on stability, so a curated selection of tested, mature packages enables the creation of very robust systems [9].

However, this does not at all fit within the Linux From Scratch philosophy, which is about users creating their own system as they please. *Itch* is simply intended to be a tool to help users with this, and therefore it should not limit the range of software users can install, or curate package repositories based on release

version. This choice should be fully in the hands of the user.

Furthermore, since this relies on an online repository of curated software, it violates Aim [5](#).

2.3.2 Holding a package database

3 Programming the package manager

References

- [1] Richard Stallman. *Linux and the GNU System*. 2nd Nov. 2021. URL: <https://www.gnu.org/gnu/linux-and-gnu.en.html> (visited on 26/08/2025).
- [2] Arch Linux contributors. *pacman*. 6th Sept. 2025. URL: <https://wiki.archlinux.org/title/Pacman> (visited on 07/09/2025).
- [3] Linux From Scratch. *Welcome to Linux From Scratch!* 2025. URL: <https://www.linuxfromscratch.org/> (visited on 24/08/2025).
- [4] Linux From Scratch. *Package Management*. 5th Mar. 2025. URL: <https://www.linuxfromscratch.org/lfs/view/stable/chapter08/pkgmgt.html> (visited on 24/08/2025).
- [5] Ben Van Daele. *Pacman on LFS 8.1*. 1st Aug. 2019. URL: <https://github.com/benvd/lfs-pacman?tab=readme-ov-file> (visited on 24/08/2025).
- [6] Reddit. *I'm curious as to how many linux users can code*. 2022. URL: <https://archive.is/2tcxS> (visited on 07/09/2025).
- [7] Pietro Abate et al. *Dependency Solving: a Separate Concern in Component Evolution Management*. Univ Paris Diderot and INRIA Paris-Rocquencourt, 2012.
- [8] Arch Linux contributors. *Arch Linux*. 3rd Sept. 2025. URL: https://wiki.archlinux.org/title/Arch_Linux (visited on 20/09/2025).
- [9] Debian contributors. *WhyDebian*. 3rd July 2022. URL: https://wiki.debian.org/WhyDebian#Utility_and_usability (visited on 20/09/2025).